

# Autonomous Research and Competitor Tracker

<https://gemini.google.com/app/075d24308a223321>

*User prompt: Problem Statement: Organizations, startups, and research institutions operate in highly competitive and rapidly evolving environments where staying updated on research trends, patent developments, competitor strategies, and industry news is critical. However, manually monitoring scientific publications, patent databases, news platforms, and social media sources is time-consuming, inefficient, and prone to missing important updates. The lack of timely insights can result in lost opportunities, delayed innovation, and weakened competitive positioning. Therefore, there is a need for an autonomous AI agent capable of continuously tracking research and competitor activities, analyzing vast information sources, and delivering concise, actionable insights in real time. Theme: Research & Competitor Tracking What is Expected: Participants are expected to build an autonomous AI agent that can continuously gather, analyze, and prioritize information from research, patents, competitors, news, and other sources, and deliver concise, actionable insights. The solution should demonstrate genuine agentic behaviour, not merely a basic scraper or information aggregator. give the project directory structure for this project and give the what is project what need to make*

Response: This is an excellent problem statement that demands a shift from traditional data engineering (cron jobs + web scraping) to a **modern AI agent architecture**. To meet the requirement of "genuine agentic behavior," your system needs autonomy, memory, reasoning, and the ability to use tools dynamically.

Here is a breakdown of what this project actually is, what you need to build, and a modular directory structure to organize it.

## 1. What the Project Is: "The Autonomous Analyst"

At its core, you are building an **Autonomous Multi-Agent System**. Instead of simply writing code that runs every 24 hours to fetch RSS feeds and summarize them, you are creating digital employees that collaborate.

To prove genuine agentic behavior, your project must demonstrate:

- **Goal-Oriented Autonomy:** You give it a high-level goal (e.g., *"Monitor solid-state battery patents and track Competitor X's progress"*), and the agent figures out *which* search queries to run, which APIs to hit, and what to click.
- **Self-Correction & Reasoning:** If a search tool returns useless data, the agent should realize this, refine its search query, and try again without human intervention.
- **State & Memory:** The agent needs to remember what it reported yesterday so it doesn't spam the user with duplicate news today. It should connect new findings to past findings (e.g., *"Competitor X patented this today, which aligns with the research paper they published 3 months ago"*).
- **Insight Prioritization:** It shouldn't just dump a massive summary. It must score the findings based on urgency or impact, surfacing critical threats or opportunities immediately while logging minor updates for a weekly digest.

## 2. What Needs to Be Made

To build this, you should adopt a **Multi-Agent Workflow** (using frameworks like CrewAI, AutoGen, or LangGraph). Here are the core components you need to develop:

## The Agents (The Brains)

- **The Scout (Data Gatherer):** Equipped with tools to search Google, query the ArXiv API (for papers), the USPTO API (for patents), and scrape news sites.
- **The Analyst (Synthesizer):** Reads the raw data gathered by the Scout. It filters out noise, connects dots, and identifies genuine trends.
- **The Editor (Decision Maker):** Reviews the Analyst's insights, checks the memory database to ensure it's new information, assigns an "Impact Score" (1-10), and decides whether to send a real-time Slack/Email alert or save it for a daily report.

## The Tools (The Hands)

- **Web Scraper / Search API:** SerpAPI or Tavily for internet searches.
- **Domain-Specific APIs:** Semantic Scholar (papers), Google Patents, Yahoo Finance (market news).
- **Notification Integration:** Slack, Microsoft Teams, or Resend (email).

## The Memory (The Storage)

- **Vector Database:** (e.g., Pinecone, ChromaDB, or Qdrant) to store past reports and raw data as embeddings. This allows the agents to perform semantic searches on historical data to spot long-term trends.
- **Relational Database:** (e.g., PostgreSQL/SQLite) to store user configurations, targeted keywords, competitor lists, and scheduled tasks.

## 3. Project Directory Structure

Here is a production-ready, modular directory structure for a Python-based multi-agent backend (using a framework like CrewAI or LangGraph) and a lightweight frontend dashboard.

```
research-competitor-agent/
├── .env                    # Environment variables (API keys, DB URIs)
├── docker-compose.yml     # For running the DBs and App locally
├── requirements.txt       # Python dependencies
├── README.md              # Project documentation
├── frontend/              # (Optional) Next.js or Streamlit UI for user dashboard
│   ├── app/               # UI pages (Dashboard, Config, Alerts)
│   └── package.json
├── backend/               # Core AI Agent Logic
│   └── main.py             # FastAPI entry point (Starts API server)
```

|                        |                                                         |
|------------------------|---------------------------------------------------------|
| — scheduler.py         | # Background task runner (triggers agents periodically) |
| — api/                 | # API endpoints for the frontend                        |
| — routes.py            | # Endpoints to add competitors, view insights, etc.     |
| — models.py            | # Pydantic schemas for API validation                   |
| — agents/              | # Define the roles, goals, and backstories of agents    |
| — __init__.py          |                                                         |
| — scout_agent.py       | # Searches and gathers data                             |
| — analyst_agent.py     | # Analyzes and correlates data                          |
| — editor_agent.py      | # Prioritizes and formats output                        |
| — tasks/               | # Define what the agents actually do                    |
| — __init__.py          |                                                         |
| — gathering_tasks.py   | # Instructions for the scout                            |
| — analysis_tasks.py    | # Instructions for the analyst                          |
| — workflows/           | # Orchestration: Tying agents and tasks together        |
| — __init__.py          |                                                         |
| — crew_orchestrator.py | # e.g., CrewAI or LangGraph state graph setup           |
| — tools/               | # Custom tools the agents can call                      |
| — __init__.py          |                                                         |
| — web_search.py        | # Web search tool integration                           |
| — patent_api.py        | # Patent database scraper/API                           |
| — arxiv_api.py         | # Academic paper fetcher                                |
| — notifier.py          | # Slack/Email tool for urgent insights                  |
| — memory/              | # RAG and long-term memory management                   |
| — __init__.py          |                                                         |
| — vector_db.py         | # Chroma/Pinecone connection & CRUD                     |
| — rag_utils.py         | # Functions to embed and retrieve past context          |
| — config/              | # Configuration files                                   |
| — settings.py          | # Loads .env and handles global settings                |
| — prompts.yaml         | # Centralized file for system prompts (optional)        |

User prompt: <https://chatgpt.com/share/6a893427-46d0-83e8-818c-8518afa50712> this the chatgpt link of the analyze this link and how to push the on git for the acesss for the other

Response: I cannot read the content inside that ChatGPT share link because the site's layout prevents automated tools from extracting the chat text. If you paste the specific code, architecture, or details from that conversation here, I can analyze it for you!

In the meantime, if you have the code ready and want to push it to a platform like GitHub so your team or collaborators can access it, follow these steps:

## 1 Create a Remote Repository

On GitHub, GitLab, or Bitbucket

Log into your Git platform and create a new, empty repository. Do not initialize it with a README, .gitignore, or license just yet to avoid initial merge conflicts.

## 2 Initialize Git Locally

Open your terminal, navigate to your project directory, and initialize the local repository:

```
cd path/to/your/project
git init
```

## 3 Stage and Commit Your Files

Add all your project files and commit them:

```
git add .
git commit -m "Initial commit: Base project setup"
```

## 4 Link the Remote Repository

Connect your local code to the empty repository you created in Step 1. (Replace the URL with your actual repository link).

```
git branch -M main
git remote add origin https://github.com/yourusername/your-repo-name.git
```

## 5 Push to Remote

Upload your code to make it accessible to others:

```
git push -u origin main
```

Once pushed, you can share the repository URL with others. If it is a private repository, you will need to navigate to the repository settings and manually invite them as collaborators using their usernames or email addresses.

---

*User prompt: divide this directory work on 4 person*

Response: Dividing this project among four people works best when you split it by **domain responsibility** rather than just cutting the folder in half. This minimizes Git merge conflicts and allows everyone to work on their specific piece in parallel.

Here is the ideal breakdown for a 4-person team:

### Person 1: Full-Stack & API Lead

**Focus:** User interaction, API endpoints, and system deployment. They ensure the user can actually interact with the AI agents.

- **frontend/** - Building the dashboard (e.g., using Streamlit) to display insights, configure alerts, and input search queries.

- **backend/main.py** - Setting up the FastAPI server.
- **backend/api/** - Creating the endpoints (routes and schemas) that connect the frontend to the agent orchestration.
- **docker-compose.yml** - Containerizing the app so it runs consistently on everyone's machine.

## Person 2: Agent Architecture Lead (AI Engineer)

**Focus:** The "brains" and the workflow. They design the prompts, define the agent personas, and connect them in a logical chain (using LangGraph or CrewAI).

- **backend/agents/** - Coding the Scout, Analyst, and Editor agents.
- **backend/tasks/** - Defining the specific instructions and expected outputs for each agent.
- **backend/workflows/** - Writing the orchestration logic that passes the Scout's output to the Analyst, and the Analyst's output to the Editor.
- **backend/config/prompts.yaml** - Managing and tuning the system prompts to prevent hallucinations.

## Person 3: Integration & Tooling Lead

**Focus:** The "hands" of the system. They build the custom tools the agents will call to interact with the outside world.

- **backend/tools/web\_search.py** - Integrating SerpAPI/Tavily for live web searches.
- **backend/tools/arxiv\_api.py** & **patent\_api.py** - Fetching academic papers and patent data.
- **backend/tools/notifier.py** - Setting up the Slack, Teams, or Email integrations for the Editor agent to send alerts.
- **backend/scheduler.py** - Setting up the background cron jobs that trigger the multi-agent workflow automatically.

## Person 4: Memory & RAG Specialist (Data Engineer)

**Focus:** The long-term memory. They ensure the agents don't report the same news twice and can look up historical data.

- **backend/memory/vector\_db.py** - Setting up ChromaDB, Pinecone, or Qdrant to store past findings.
- **backend/memory/rag\_utils.py** - Writing the functions to embed text into vectors and retrieve relevant past context for the Analyst agent.
- **backend/config/settings.py** - Managing database connections and configuration state.
- **.env** - Managing all the API keys (OpenAI, Vector DB, Search APIs) securely.

## How to avoid blocking each other on Day 1:

To start coding immediately without waiting for others to finish their parts, use **mock data**:

1. **Person 1** should build the UI using fake JSON data while waiting for Person 2's workflows.
  2. **Person 2** should test their agents using hardcoded Python strings instead of waiting for Person 3's real APIs.
  3. **Person 3** can test their API scripts independently in terminal scripts.
  4. **Person 4** can start embedding sample text documents into the Vector DB to test retrieval.
-